

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
SPECIALIZED PROJECT (CO4029)

Nghiên cứu và xây dựng Tác tử AI Tự hành để Đánh giá Mức độ Trưởng thành Bảo mật trên Nền tảng AWS

Major: Computer Science

Supervisors:	Nguyen An Khuong, PhD Pham Nhut Huy
Students:	Võ Nguyễn Đức Phát – Student ID: 2212540 Nguyễn Trung Phú – Student ID: 2153289

Ho Chi Minh City, November 2025

PhD Nguyen An Khuong (Supervisor)

Date: _____

Declaration

We hereby declare that this research is entirely our own work, conducted under the supervision and guidance of Dr. Nguyen An Khuong and Mr. Pham Nhut Huy. The results presented in this research are legitimate and have not been published in any form prior to this. All materials used in this research have been collected by us from various sources and are properly cited in the bibliography section. Additionally, all figures included in this report have been created by us.

This research also references results from other authors, which have already been published by the respective authors.

In the event of any plagiarism, we take full responsibility for our actions. Ho Chi Minh City University of Technology - Vietnam National University is not liable for any copyright infringements arising from this research.

Ho Chi Minh City, November 2025

Authors

Acknowledgements

Firstly, we would like to sincerely express our gratitude to our supervisors, Dr. Nguyen An Khuong, and Mr. Pham Nhut Huy. To Dr. Nguyen An Khuong, thank you for dedicating your precious time to us. The weekly meetings helped our team stay on track, and your timely feedback, advice, and guidance greatly contributed to the success of our research. To Mr. Pham Nhut Huy, your expertise in the field, along with your thesis writing and presentation skills, was invaluable to our progress.

Secondly, we wish to extend our heartfelt thanks to our families for their unwavering support, both emotionally and financially, throughout this research process. Our accomplishments would not have been possible without their encouragement and assistance.

Lastly, this is from Nguyen Trang Sy Lam, the leader of the team. I would like to personally thank Dr. Nguyen An Khuong for guiding me through the most challenging days of my student life. When I was in a dark place, your support and mentorship pulled me through and helped me complete this research. Words cannot fully convey my gratitude to you.

Abstract

In this thesis, we present the design of a Retrieval-Augmented Generation (RAG) pipeline integrated into a Visual Studio Code (VSCode) extension, serving as an intelligent assistant that run locally for web developers working with the Next.js framework. The pipeline leverages Milvus [2] as the vector database to efficiently store and retrieve information from Next.js documentation, prepared during the data processing phase. For response generation, the system utilizes Alibaba's Qwen [1], a Large Language Model (LLM). This combination of technologies ensures the system providing accurate, context-aware assistance. By retrieving relevant information and generating responses grounded in the retrieved data, the proposed design enhances productivity and simplifies the development process within the VSCode environment.

Mục lục

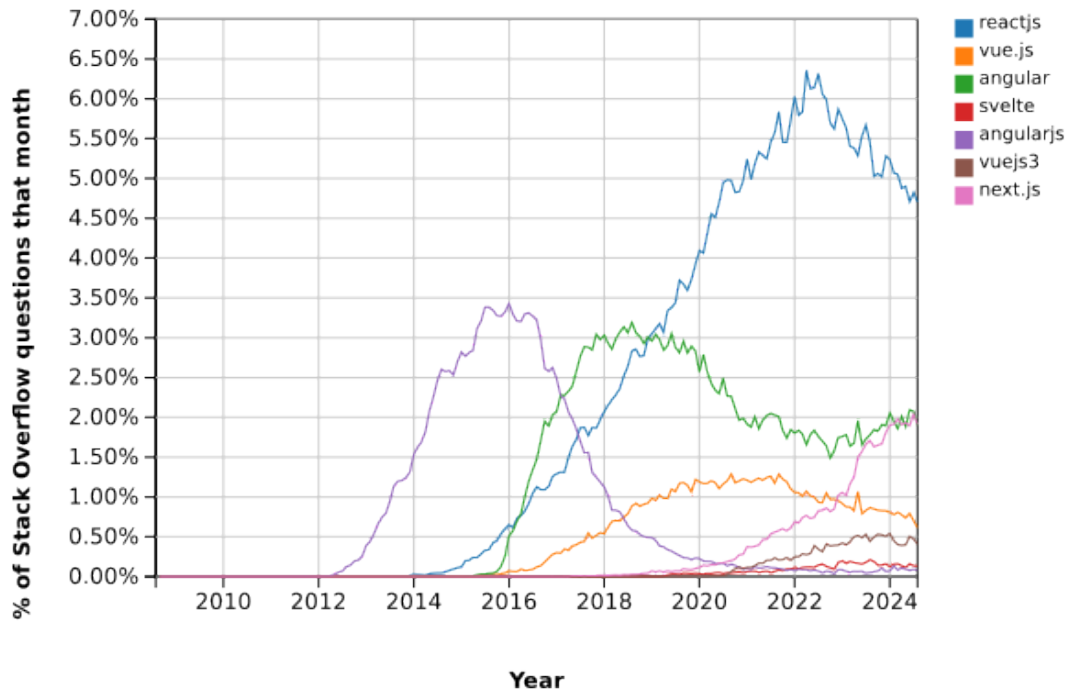
1.1 Motivations

Sự bùng nổ của điện toán đám mây, đặc biệt là nền tảng Amazon Web Services (AWS), đã mang lại khả năng mở rộng và linh hoạt chưa từng có cho các doanh nghiệp. Tuy nhiên, đi kèm với lợi ích đó là sự gia tăng phức tạp trong việc quản lý hạ tầng và đảm bảo an ninh. Các công cụ đánh giá bảo mật đám mây (Cloud Security Posture Management - CSPM) hiện nay như Prowler hay ScoutSuite đóng vai trò quan trọng trong việc phát hiện các lỗ hổng cấu hình. Mặc dù rất mạnh mẽ trong việc chỉ ra các sai sót kỹ thuật, nhưng kết quả đầu ra của chúng thường dừng lại ở các báo cáo dạng văn bản hoặc dữ liệu thô (JSON, CSV). Điều này tạo ra một gánh nặng lớn cho các kỹ sư vận hành, khi họ phải thủ công phân tích từng cảnh báo, đối chiếu tài liệu và tự thực hiện các thao tác sửa lỗi (remediation) lặp đi lặp lại.

Trong khi đó, các Mô hình Ngôn ngữ Lớn (LLM) đã cho thấy khả năng vượt trội trong việc hiểu và xử lý ngôn ngữ tự nhiên. Tuy nhiên, nếu chỉ sử dụng LLM như một công cụ tư vấn (chatbot) thụ động thì chưa đủ để giải quyết bài toán vận hành thực tế, vì chúng thiếu khả năng tương tác trực tiếp với môi trường để thực thi hành động.

Để giải quyết khoảng cách giữa việc "phát hiện lỗi" và "xử lý lỗi", đề án này đề xuất nghiên cứu và xây dựng một hệ thống Tác tử AI Tự hành (Autonomous AI Agent) dựa trên kiến trúc Đa tác tử (Multi-agent system) kết hợp với kỹ thuật Gọi công cụ (Tool Calling). Thay vì chỉ đưa ra lời khuyên, hệ thống này được thiết kế để hoạt động như một đội ngũ kỹ sư ảo, trong đó các tác tử chuyên biệt phối hợp với nhau để thực hiện nhiệm vụ. Điểm đột phá của phương pháp này nằm ở khả năng "Tool Calling" – cho phép LLM không chỉ sinh văn bản mà còn có thể ra quyết định gọi các lệnh thực thi (như AWS CLI, Prowler CLI) để thao tác trực tiếp trên hạ tầng đám mây.

Cách tiếp cận này cho phép hệ thống tự động hóa trọn vẹn quy trình đánh giá bảo mật: từ việc chủ động quét môi trường để phát hiện rủi ro, phân tích mức độ nghiêm trọng, cho đến việc tự động thực thi các hành động khắc phục (remediation) để vá lỗ hổng ngay lập tức mà không cần sự can thiệp thủ công của con người. Đây là bước tiến quan trọng nhằm chuyển đổi quy trình bảo mật từ mô hình thụ động sang mô hình chủ động và tự hành hoàn toàn.



Hình 1.1: The popularity of frameworks based on StackOverflow questions 2008-2024

To illustrate, consider the Next.js framework, which has gained significant popularity in recent years. Built on top of React.js (currently the most popular front-end library), Next.js has seen rapid growth and frequent updates. For instance, version 13.0.0 of Next.js was released on October 25th, 2022, followed by multiple version 13.x.x updates, until version 14.0.0 was released on October 26th, 2023. Just 1 year later, they released version 15.0.0 for the framework with many breaking changes¹. This rapid pace of development introduces challenges for developers who need to maintain or upgrade their codebases, especially considering that React.js itself has also undergone significant changes, such as the switch from version 18.0.0 to 19.0.0, which involved major updates².

1.2 Objectives

Mục tiêu chính của nghiên cứu này là thiết kế và xây dựng một hệ thống Tác tử AI Tự hành (Autonomous AI Agent) có khả năng đánh giá, báo cáo và nâng cao mức độ trưởng thành về bảo mật trên nền tảng AWS. Bằng cách tận dụng kiến trúc **Đa tác tử (Multi-agent systems)** và kỹ thuật **Gọi công cụ (Tool Calling)**, đề án hướng tới các mục tiêu cụ thể sau:

- Xây dựng kiến trúc đa tác tử để tự động hóa quy trình đánh giá bảo mật thông qua việc tương tác với các công cụ quét tiêu chuẩn (như Prowler). Hệ thống sẽ phân tích kết quả thô và tổng hợp thành các **báo cáo đánh giá toàn diện** (report generation), cung cấp cái nhìn sâu sắc, định tính về hiện trạng an ninh và mức độ tuân thủ của môi trường AWS, giúp người quản trị dễ dàng nắm bắt rủi ro.
- Hiện thực hóa cơ chế **tự động khắc phục** (autonomous remediation). Không chỉ dừng lại ở việc tạo báo cáo khuyến nghị, tác tử sẽ có khả năng chủ động lập kế hoạch và thực thi các lệnh cần thiết (thông qua AWS CLI) để trực tiếp sửa chữa các lỗi cấu hình bảo mật, giúp khép kín quy trình từ phát hiện, báo cáo đến xử lý triệt để.

Nghiên cứu này nhằm chứng minh tính khả thi của việc chuyển đổi hoạt động vận hành bảo mật đám mây từ quy trình thủ công sang một hệ thống chủ động, minh bạch trong báo cáo và có khả năng tự phục hồi (self-healing).

¹The latest Next.js news - <https://nextjs.org/blog>

²How to Upgrade to React 18 - <https://react.dev/blog/2024/04/25/react-19-upgrade-guide>

1.3 Scope

Phạm vi của đề án tập trung vào việc phát triển một hệ thống Multi-Agent tự động hóa quy trình đánh giá và nâng cao mức độ trưởng thành về bảo mật cho nền tảng điện toán đám mây AWS (Amazon Web Services). Hệ thống được thiết kế để hoạt động như một giải pháp kiểm toán và khắc phục sự cố an ninh tích hợp, bao gồm các giới hạn và đặc điểm kỹ thuật sau:

- **Môi trường Đám mây Mục tiêu:** Hệ thống tập trung vào việc đánh giá và bảo vệ các dịch vụ AWS cốt lõi, bao gồm nhưng không giới hạn ở Identity and Access Management (IAM), Simple Storage Service (S3), và Elastic Compute Cloud (EC2).
- **Công nghệ AI và LLM:** Sử dụng các mô hình ngôn ngữ lớn (LLMs) mã nguồn mở, cụ thể là Llama 3.2, được triển khai cục bộ thông qua nền tảng Ollama để đảm bảo quyền riêng tư dữ liệu và khả năng tùy biến hành vi của các tác nhân (agents).
- **Kiến trúc Multi-Agent:** Hệ thống được xây dựng dựa trên kiến trúc đa tác nhân chuyên biệt, bao gồm các thành phần chính:
 - *Planning Agent:* Phân tích yêu cầu người dùng và lập kế hoạch kiểm tra.
 - *Task Assignment Agent:* Điều phối và thực thi các tác vụ quét song song.
 - *Scanner Agent:* Tương tác với các công cụ quét bảo mật.
 - *Monitoring Agent:* Theo dõi tiến độ và thu thập dữ liệu.
 - *Remediate Agent:* Phân tích kết quả và đề xuất hoặc thực thi các biện pháp khắc phục tự động.
 - *Reporting Agent:* Tổng hợp và báo cáo kết quả cuối cùng.
 - *Assessment Agent:* Đánh giá mức độ tuân thủ so với các tiêu chuẩn bảo mật (Security Maturity Model) sử dụng kỹ thuật RAG (Retrieval-Augmented Generation).
- **Công cụ Tích hợp:** Hệ thống tích hợp với Prowler (phiên bản 3 trở lên) làm công cụ quét bảo mật nền tảng (security scanning engine) và sử dụng AWS SDK cho Python (Boto3) để thực hiện các tác vụ khắc phục sự cố tự động.
- **Ngôn ngữ và Framework:** Toàn bộ hệ thống backend và logic điều phối agent được phát triển bằng ngôn ngữ Python, sử dụng FastAPI để xây dựng API Server và thư viện 'threading' để xử lý đa luồng.

Phạm vi này loại trừ việc phát triển giao diện người dùng (frontend) phức tạp, thay vào đó tập trung vào logic xử lý backend, khả năng tương tác giữa các agent AI, và tính chính xác trong việc phát hiện cũng như khắc phục các lỗ hổng bảo mật trên AWS.

1.4 Challenges

Developing this tool involves several challenges:

- **Efficient Context Management:** Since the goal is to make the tool lightweight, storing and managing the context of a user's codebase in a way that allows for fast retrieval without overwhelming system resources is a major challenge.
- **Data Curation Complexity:** The data curation process requires extensive knowledge about the framework.
- **Handling Framework Updates:** Keeping the tool updated with the frequent changes in frameworks like Next.js and their associated packages is challenging. Continuous updates are needed to ensure compatibility and provide accurate suggestions.
- **User Experience:** Balancing the amount of information presented to the user without overwhelming them is essential. The tool should provide meaningful insights and recommendations without causing information overload.

1.5 Structure of the report

Chapter 1. Introduction. This chapter introduces the motivations, objectives, and scope of the research. It highlights the challenges of web development and the role of RAG in enhancing efficiency and accuracy. The chapter outlines the structure of the report, providing a roadmap for the research presented.

Chapter 2. Preliminaries. This chapter provides foundational knowledge on RAG, emphasizing its retriever and generator components. It discusses indexing, vector embeddings, and augmentation methods, offering a clear understanding of the technologies used. Key concepts of the Next.js framework and associated development tools are also introduced.

Chapter 3. Related Works and Tools. This chapter explores prior research on RAG and its application in web development. It reviews developer tools such as VSCode extensions and discusses strategies for retrieval and augmentation. Comparative analyses of existing frameworks and methodologies are also presented.

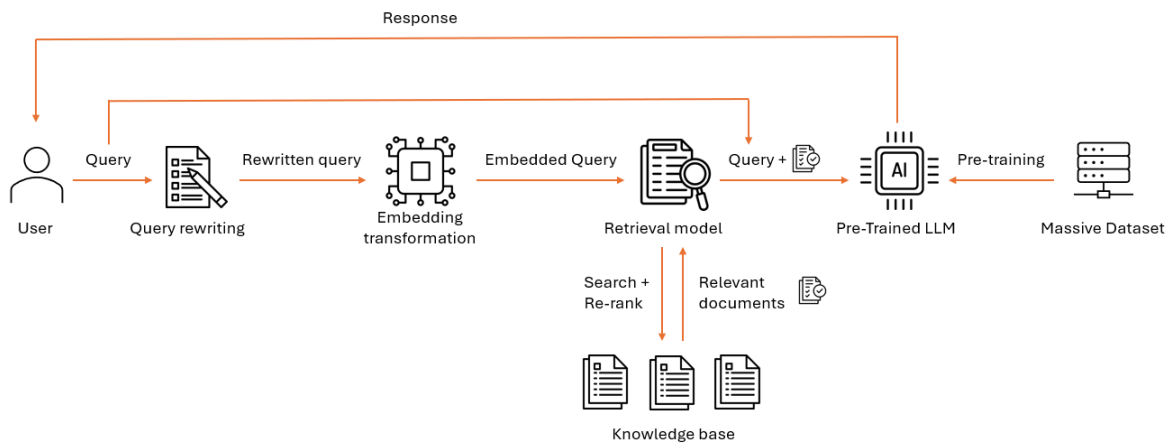
Chapter 4. Approach. This chapter describes the proposed solution, including the architecture of the RAG-enhanced system. It explains the rationale for selecting specific models like BGE-M3 and Qwen and details the integration of local vector storage with the Next.js framework to ensure efficient retrieval and generation.

Chapter 5. Initial implementation. This chapter outlines the step-by-step development process of the RAG pipeline. It includes data preparation, database setup, and integration with the VSCode extension. Challenges encountered during implementation and their solutions are also discussed.

Chapter 6. Evaluation plan. This chapter evaluates the system using metrics such as accuracy and developer productivity. It describes the testing methodology, compares results with existing solutions, and highlights the strengths and limitations of the proposed approach.

Chapter 7. Future plan. The final chapter summarizes the contributions of the research and discusses potential areas for improvement. It offers recommendations for extending the system's compatibility and incorporating advanced retrieval techniques to enhance performance and scalability.

Chương này trình bày các khái niệm nền tảng và công nghệ cốt lõi được sử dụng trong việc xây dựng hệ thống. Nội dung bao gồm tổng quan về các mô hình ngôn ngữ, kỹ thuật tích hợp tri thức ngoài (RAG), và kiến trúc hệ thống đa tác nhân (Multi-Agent Systems).



Hình 2.1: Retrieval-Augmented Generation Process

The RAG process is composed of two main components: the retriever and the generator. The retriever is responsible for locating and extracting relevant information from an external knowledge source, employing techniques such as keyword-based search, semantic similarity search, or neural network-based retrieval to ensure precise results. The generator then utilizes the retrieved context to produce coherent and accurate responses, ensuring the output aligns with the user's query while maintaining relevance and context. Together, these components form a robust framework for leveraging external knowledge in generative tasks.

2.1 Mô hình Ngôn ngữ Lớn và Nhỏ (LLMs & SLMs)

2.1.1 Sự chuyển dịch từ LLMs sang SLMs

Trong những năm gần đây, lĩnh vực Trí tuệ Nhân tạo (AI) đã chứng kiến sự bùng nổ của các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) như GPT-4, Claude 3, hay Gemini. Các mô hình này sở hữu hàng trăm tỷ tham số (parameters), cho phép chúng thực hiện các tác vụ phức tạp với độ chính xác cao. Tuy nhiên, việc triển khai các LLMs này đòi hỏi tài nguyên tính toán khổng lồ (thường là các cụm GPU H100/A100 trong các trung tâm dữ liệu), dẫn đến chi phí vận hành cao và độ trễ mạng đáng kể.

Để giải quyết các hạn chế này, xu hướng phát triển các Mô hình Ngôn ngữ Nhỏ (Small Language Models - SLMs) đã nổi lên như một giải pháp tối ưu cho các ứng dụng biên (edge computing) và cục bộ (on-premise). Các mô hình như **Llama 3.2** (với kích thước 1B và 3B tham số) đại diện cho thế hệ SLM mới, được tối ưu hóa để cân bằng giữa hiệu năng và tài nguyên.

Ưu điểm nổi bật của việc sử dụng SLMs trong đồ án này bao gồm:

- **Quyền riêng tư dữ liệu (Data Privacy):** Toàn bộ dữ liệu nhạy cảm (như thông tin cấu hình AWS, logs bảo mật) được xử lý cục bộ trên máy tính cá nhân, không bao giờ được gửi ra ngoài internet tới các máy chủ của bên thứ ba.
- **Hiệu quả chi phí và năng lượng:** SLMs có thể vận hành mượt mà trên các thiết bị có phần cứng hạn chế (CPU hoặc GPU laptop thông thường), giảm thiểu chi phí hạ tầng.
- **Khả năng tùy biến cao:** Với kích thước nhỏ gọn, SLMs dễ dàng được tinh chỉnh (fine-tune) cho các tác vụ chuyên biệt như kiểm toán bảo mật mà không tốn quá nhiều thời gian huấn luyện.

2.1.2 Mô hình Ngôn ngữ Nhỏ (SLM): Meta Llama 3.2

Để đảm bảo cân bằng giữa hiệu năng suy luận và tài nguyên hệ thống cục bộ, chúng tôi lựa chọn ****Llama 3.2**** (phiên bản 3 tỷ tham số - 3B) làm mô hình trí tuệ nhân tạo nòng cốt cho các Agent. Đây là dòng mô hình ngôn ngữ nhỏ (Small Language Model - SLM) thế hệ mới nhất của Meta, được phát hành vào tháng 9/2024.

Lý do lựa chọn Llama 3.2 3B cho hệ thống kiểm toán bảo mật bao gồm:

- **Tối ưu hóa cho Tác vụ Agent (Agentic Capabilities):** Khác với các mô hình ngôn ngữ truyền thống chỉ mạnh về hội thoại, Llama 3.2 được tinh chỉnh (instruction-tuned) đặc biệt cho các tác vụ "Agentic Retrieval" và "Tool Use". Điều này cho phép mô hình hiểu và ra quyết định gọi các công cụ bên ngoài (như Prowler, AWS CLI) một cách chính xác hơn so với các đối thủ cùng phân khúc.
- **Hiệu suất trên Tài nguyên thấp:** Với kỹ thuật lượng tử hóa (Quantization) chuẩn Q4_K_M, mô hình chỉ chiếm khoảng **2.0GB** bộ nhớ VRAM/RAM. Điều này cho phép chạy song song nhiều Agent (Scanner, Risk Eval, Remediation) trên một máy tính cá nhân mà không gặp hiện tượng nghẽn cổ chai về phần cứng.
- **Khả năng Suy luận vượt trội:** Theo các bảng xếp hạng công nghiệp (Industry Benchmarks), phiên bản 3B này vượt trội hơn so với Google Gemma 2 (2.6B) và Microsoft Phi-3.5 mini ở các tác vụ:
 - Tuân thủ chỉ thị phức tạp (Following instructions).
 - Tóm tắt nội dung dài (Summarization) - hữu ích cho việc tạo báo cáo.
 - Viết lại Prompt (Prompt rewriting) - hữu ích cho việc tinh chỉnh luồng suy luận CoT.
- **Hỗ trợ đa ngôn ngữ:** Llama 3.2 hỗ trợ chính thức 8 ngôn ngữ và được huấn luyện trên tập dữ liệu rộng hơn, giúp cải thiện khả năng xử lý các báo cáo hoặc tài liệu đan xen tiếng Anh chuyên ngành và tiếng Việt.

images/llama32_benchmark_comparison.png

Hình 2.2: So sánh hiệu năng giữa Llama 3.2 3B với Gemma 2 2.6B và Phi-3.5 Mini trên các tác vụ quan trọng (Nguồn: Meta AI Research)

Thông số kỹ thuật của mô hình được triển khai:

Thuộc tính	Giá trị
Kiến trúc (Architecture)	Llama
Số lượng tham số	3.21 Tỷ (3.21B)
Phương pháp lượng tử hóa	Q4_K_M (4-bit)
Dung lượng mô hình	~ 2.0 GB
Context Window	128k token
License	Llama 3.2 Community License

Bảng 2.1: Thông số kỹ thuật của Meta Llama 3.2 3B chạy trên Ollama

2.1.3 Lượng tử hóa

Để triển khai các mô hình ngôn ngữ lớn (LLM) trên hạ tầng phần cứng giới hạn (Consumer Hardware), chúng tôi áp dụng kỹ thuật **Lượng tử hóa (Quantization)**. Đây là quá trình ánh xạ các tham số của mô hình từ miền giá trị có độ chính xác cao (thường là FP16 - 16-bit Floating Point) sang miền giá trị có độ chính xác thấp hơn (như INT8 hoặc INT4) nhằm giảm thiểu kích thước bộ nhớ và tăng tốc độ tính toán.

2.1.3.1 Định dạng GGUF và K-Quants

Trong đồ án này, chúng tôi sử dụng định dạng **GGUF (GPT-Generated Unified Format)**. Đây là định dạng tệp nhị phân được tối ưu hóa cho thư viện `llama.cpp`, mang lại hai lợi ích cốt lõi:

1. **Memory Mapping (mmap):** GGUF cho phép hệ điều hành ánh xạ tệp mô hình trực tiếp vào

bộ nhớ ảo mà không cần tải toàn bộ vào RAM ngay lập tức, giúp khởi động mô hình cực nhanh và chia sẻ bộ nhớ hiệu quả giữa CPU/GPU.

2. **K-Quants (k-means quantization):** Llama 3.2 được lượng tử hóa theo phương pháp Q4_K_M. Khác với lượng tử hóa tuyến tính thông thường, K-Quants chia các ma trận trọng số (weights) thành các khối nhỏ (super-blocks) và sử dụng thuật toán tối ưu hóa để gán bit quan trọng cho các tham số trọng yếu (như lớp *Attention*) và giảm bit ở các lớp ít quan trọng hơn (như *Feed-forward*).

Hiệu quả thực nghiệm: Việc áp dụng mức lượng tử hóa Q4_K_M cho Llama 3.2 3B giúp giảm kích thước mô hình từ ~ 6.5 GB (FP16) xuống còn ~ 2.0 GB, cho phép vận hành trơn tru trên các máy tính cá nhân có RAM 8GB mà vẫn giữ được 98-99% hiệu năng (perplexity score) so với bản gốc.

2.1.4 Nền tảng Ollama và Kiến trúc suy luận cục bộ

Ollama là một khung làm việc (framework) mã nguồn mở giúp đơn giản hóa việc chạy các LLM/SLM cục bộ trên macOS, Linux và Windows. Ollama đóng vai trò như một lớp trừu tượng hóa (abstraction layer), quản lý việc tải mô hình, quản lý bộ nhớ, và cung cấp giao diện API tương thích với chuẩn OpenAI.

Kiến trúc của Ollama bao gồm các thành phần chính:

- **Ollama Server:** Một tiến trình nền (daemon) chịu trách nhiệm quản lý vòng đời của mô hình và xử lý các yêu cầu HTTP từ client.
- **Modelfile:** Một tệp cấu hình định nghĩa các tham số của mô hình, bao gồm **SYSTEM PROMPT** (lời nhắc hệ thống), nhiệt độ (temperature), và các tham số lấy mẫu khác. Đây là nơi chúng em lập trình hành vi cho từng Agent chuyên biệt (như Planning Agent, Remediation Agent).
- **REST API:** Cung cấp các điểm cuối (endpoints) như `/api/generate` và `/api/chat` để các ứng dụng Python có thể tương tác với mô hình một cách dễ dàng.

2.1.5 Kỹ thuật Prompt Engineering và Fine-tuning

Để tối ưu hóa hiệu suất của Llama 3.2 cho tác vụ kiểm toán bảo mật, đề án áp dụng kỹ thuật chính:

1. **Chain-of-Thought (CoT) Prompting:** Trong kiến trúc Multi-Agent của đề án, chúng em áp dụng CoT để khắc phục hạn chế của việc lập trình các quy tắc cứng (hardcoding). Kỹ thuật này cho phép các tác nhân (agents) thực hiện suy luận từng bước dựa trên dữ liệu đầu vào, giúp hệ thống tự động thích nghi và đưa ra các quyết định phù hợp với ngữ cảnh động thay vì tuân theo các kịch bản định sẵn.

2.2 Framework LangChain

Để xây dựng nền tảng cho các tác nhân AI (Agents), chúng em sử dụng **LangChain** - một khung làm việc (framework) mã nguồn mở giúp đơn giản hóa việc phát triển các ứng dụng dựa trên Mô hình Ngôn ngữ Lớn (LLM). LangChain đóng vai trò là framework kết nối logic lập trình với khả năng suy luận của mô hình Llama 3.2.

2.2.1 Vai trò trong Kiến trúc Hệ thống

Trong đề án này, LangChain không chỉ đơn thuần là công cụ gọi API, mà còn đảm nhận các nhiệm vụ cốt lõi:

- **Trừu tượng hóa Model (Model Abstraction):** Cung cấp giao diện thống nhất để tương tác với Ollama. Điều này cho phép hệ thống dễ dàng chuyển đổi giữa các mô hình (ví dụ: từ Llama 3.2 sang Mistral) mà không cần sửa đổi mã nguồn nghiệp vụ.
- **Quản lý Prompt (Prompt Engineering):** Sử dụng `PromptTemplate` để cấu trúc hóa đầu vào, kết hợp các biến động (như dữ liệu log Prowler) với các chỉ thị cố định (System Prompt).
- **Định dạng đầu ra (Output Parsing):** Chuyển đổi văn bản tự nhiên do LLM sinh ra thành các định dạng dữ liệu có cấu trúc (JSON, List) để các module phần mềm có thể xử lý tiếp.

2.2.2 LangChain Expression Language (LCEL)

Hệ thống được xây dựng dựa trên **LCEL** - một cú pháp khai báo (declarative) giúp kết nối các thành phần thành một chuỗi xử lý (chain) liên mạch. LCEL lấy cảm hứng từ toán tử "pipe" trong Unix, cho phép luồng dữ liệu di chuyển tuyến tính và minh bạch.

Một chuỗi xử lý điển hình trong đồ án được định nghĩa như sau:

$$\text{Chain} = \text{Prompt} \mid \text{LLM} \mid \text{OutputParser} \quad (2.1)$$

Trong đó:

- **Prompt:** Nhận biến đầu vào (ví dụ: `user_request`) và tạo ra đoạn văn bản hoàn chỉnh gửi cho mô hình.
- **LLM:** Thực hiện suy luận (Inference) dựa trên prompt.
- **OutputParser:** Trích xuất phần nội dung quan trọng từ phản hồi của LLM và loại bỏ các ký tự dư thừa.

Ví dụ mã nguồn minh họa việc áp dụng LCEL trong tác nhân *Planning Agent*:

```
# Minh họa cấu trúc LCEL trong mã nguồn
prompt = PromptTemplate.from_template("Lập kế hoạch quét cho dịch vụ: {service}")
llm = ChatOllama(model="llama3.2")
parser = JsonOutputParser()

# Khởi tạo chuỗi xử lý
chain = prompt | llm | parser

# Thực thi
plan = chain.invoke({"service": "AWS S3"})
```

2.2.3 Tích hợp Công cụ (Tool Integration)

LangChain cho phép mở rộng khả năng của Llama 3.2 thông qua các **Tools**. Trong hệ thống này, chúng em định nghĩa các hàm Python (như `run_proowler_scan`, `aws_cli_exec`) và bọc chúng dưới dạng LangChain Tools. Nhờ đó, Agent có thể "nhận thức" được môi trường và tác động thực tế lên hạ tầng Cloud thay vì chỉ trả lời lý thuyết suông.

2.3 Framework LangGraph

Trong khi LangChain cung cấp các khối xây dựng cơ bản (chains) hoạt động theo cơ chế đồ thị không chu trình có hướng (Directed Acyclic Graph - DAG), việc xây dựng các hệ thống tác nhân phức tạp đòi hỏi khả năng xử lý vòng lặp (loops) và duy trì trạng thái bền vững. Để giải quyết vấn đề này, chúng em sử dụng **LangGraph** - một thư viện mở rộng của LangChain chuyên dụng cho việc điều phối các ứng dụng có trạng thái (stateful) và đa tác nhân.

2.3.1 Kiến trúc Đồ thị Trạng thái (StateGraph Architecture)

LangGraph mô hình hóa luồng xử lý của ứng dụng dưới dạng một **Đồ thị Trạng thái (StateGraph)**. Khác với các chuỗi xử lý tuyến tính, kiến trúc này cho phép định nghĩa các luồng công việc có tính chu kỳ (cyclic workflows), nơi đầu ra của một bước có thể quay trở lại làm đầu vào cho bước trước đó hoặc điều hướng sang nhánh khác dựa trên logic điều kiện.

Một đồ thị trong LangGraph được định nghĩa bởi bộ ba thành phần:

$$G = (S, N, E) \quad (2.2)$$

Trong đó:

- **State (S):** Là lược đồ dữ liệu chia sẻ (Shared Schema), đóng vai trò như bộ nhớ chung cho toàn bộ đồ thị. Tất cả các Node đều truy cập và cập nhật vào State này thay vì truyền tham số trực tiếp cho nhau.
- **Nodes (N):** Là các đơn vị xử lý logic (thường là các Agent hoặc các hàm Python). Mỗi Node nhận State hiện tại làm đầu vào, thực hiện tính toán, và trả về một bản cập nhật trạng thái (State Update).
- **Edges (E):** Quy định luồng di chuyển giữa các Nodes.

2.3.2 Cơ chế Quản lý Trạng thái (State Management)

Điểm mạnh cốt lõi của LangGraph là cơ chế quản lý trạng thái tập trung. Trong đồ án này, trạng thái hệ thống được định nghĩa bằng cấu trúc `TypedDict` của Python.

Khi một Node hoàn thành tác vụ, nó không ghi đè toàn bộ trạng thái mà trả về một phần thay đổi (delta). LangGraph sẽ tự động hợp nhất (merge) thay đổi này vào trạng thái toàn cục:

$$S_{t+1} = S_t \cup \Delta S_{node} \quad (2.3)$$

Cơ chế này đảm bảo tính toàn vẹn dữ liệu khi nhiều tác nhân cùng tham gia vào quy trình xử lý (ví dụ: *Scanner Agent* cập nhật danh sách lỗi, trong khi *Risk Agent* cập nhật điểm rủi ro vào cùng một State).

2.3.3 Điều hướng Luồng và Vòng lặp (Control Flow & Cycles)

LangGraph cung cấp hai loại cạnh để điều khiển luồng đi:

1. **Cạnh thường (Normal Edges):** Chuyển đổi trực tiếp từ Node A sang Node B (ví dụ: Từ *Planning* $\rightarrow$ *Scan*).
2. **Cạnh điều kiện (Conditional Edges):** Sử dụng hàm logic để quyết định điểm đến tiếp theo dựa trên trạng thái hiện tại. Đây là nền tảng của mô hình "Human-in-the-loop" và khả năng tự sửa lỗi.

Ví dụ minh họa logic cạnh điều kiện trong hệ thống:

```
# Logic định tuyến dựa trên trạng thái
def route_after_assessment(state):
    if state["risk_level"] == "CRITICAL":
        return "remediate_node" # Chuyển sang sửa lỗi
    else:
        return "report_node"    # Chuyển sang báo cáo

workflow.add_conditional_edges("assessment", route_after_assessment)
```

Nhờ LangGraph, hệ thống có thể thực hiện các quy trình phức tạp như: *Quét* $\rightarrow$ *Đánh giá* $\rightarrow$ *Sửa lỗi* $\rightarrow$ *Quét lại* (*Rescan*). Bước "Quét lại" chính là một vòng lặp (cycle) mà các chuỗi LangChain truyền thống khó thực hiện được. section

Hệ thống được xây dựng dựa trên mô hình Đa tác nhân (Multi-Agent System - MAS) sử dụng framework **LangChain** và **LangGraph**. Thay vì xử lý tuyến tính, hệ thống quản lý trạng thái (State) và điều phối luồng công việc giữa các tác nhân chuyên biệt dưới dạng một đồ thị có hướng (State Graph).

Hệ thống bao gồm 3 tác nhân chính phối hợp với nhau để thực hiện quy trình kiểm toán và khắc phục bảo mật:

2.3.4 Cấu trúc của một Tác nhân (Agent Structure)

Trong ngữ cảnh của đồ án, mỗi tác nhân (Agent) không phải là một thực thể trừu tượng phức tạp mà được cấu thành từ 3 thành phần cơ bản:

- **Profile (Prompt):** Chỉ thị vai trò, nhiệm vụ cụ thể (ví dụ: "Bạn là chuyên gia đánh giá rủi ro...").
- **Tools (Công cụ):** Các hàm chức năng mà Agent có thể gọi (ví dụ: chạy Prowler scan, gọi AWS SDK để đổi cấu hình).

- **LLM (Brain):** Bộ não xử lý ngôn ngữ tự nhiên để quyết định xem nên dùng công cụ nào hoặc trả lời như thế nào dựa trên đầu vào.

2.3.5 Các Tác nhân Chính (Core Agents)

Hệ thống được chia nhỏ thành các tác nhân với trách nhiệm riêng biệt:

1. Planning Agent (Tác nhân lên kế hoạch):

- **Nhiệm vụ:** Tiếp nhận yêu cầu từ người dùng và tiến hành suy nghĩ đưa ra kế hoạch phù hợp.
- **Đầu ra:** Danh sách các phát hiện (findings) thô từ môi trường AWS.

2. Scanner Agent (Tác nhân Quét):

- **Nhiệm vụ:** Tiếp nhận yêu cầu từ Planning Agent và thực thi các công cụ rà soát (Trong trường hợp đồ án là: Prowler).
- **Đầu ra:** Danh sách các phát hiện (findings) thô từ môi trường AWS.

3. Risk Evaluation Agent (Tác nhân Đánh giá Rủi ro):

- **Nhiệm vụ:** Phân tích các phát hiện từ Scanner Agent. Agent này sử dụng LLM để đối chiếu với các tiêu chuẩn bảo mật, loại bỏ cảnh báo giả (false positives) và chấm điểm mức độ nghiêm trọng.
- **Đầu ra:** Danh sách các lỗi đã được xác thực và phân loại (Critical, High, Medium).

4. Remediation Agent (Tác nhân Khắc phục):

- **Nhiệm vụ:** Nhận danh sách lỗi từ Risk Evaluation Agent. Dựa trên mức độ rủi ro và cấu hình cho phép, Agent sẽ lập kế hoạch và thực thi các lệnh sửa lỗi (auto-remediation) hoặc đưa ra hướng dẫn.
- **Đầu ra:** Báo cáo kết quả khắc phục (Success/Failed).

2.3.6 Luồng hoạt động (Workflow State Management)

Chúng em sử dụng **LangGraph** để định nghĩa luồng di chuyển dữ liệu giữa các Agent, theo kiến trúc bên dưới đây. Trạng thái chung (Shared State) của hệ thống đóng vai trò như bộ nhớ chia sẻ, lưu trữ các thông tin quan trọng qua từng bước.

Quy trình xử lý được mô hình hóa đơn giản như sau:

Algorithm 1 AWS Security Audit Workflow (LangGraph Orchestration)

Input: User Request R (e.g., "Scan S3 and IAM")

Output: Final Report Path P

```

1:  $State \leftarrow \text{InitState}(R)$ 
2:  $State.Plan \leftarrow \text{PLANNINGAGENT}(R)$  ▷ Phân tích yêu cầu
3:  $JobIDs \leftarrow \text{SCANNERAGENT}(State.Plan)$ 
4:  $State.RawFindings \leftarrow \text{MONITORAGENT}(JobIDs)$  ▷ Dợi kết quả Prowler
5:  $Enriched \leftarrow \text{RISKEVALAGENT}(State.RawFindings)$  ▷ Thêm ngữ cảnh rủi ro
6:  $State.Failures, NeedFix \leftarrow \text{ASSESSMENTAGENT}(Enriched)$  ▷ Lọc lỗi vi phạm
7: if  $NeedFix$  is  $True$  then
8:    $Approval \leftarrow \text{HUMANREVIEWNODE}(State.Failures)$  ▷ Hỏi người dùng
9:   if  $Approval$  is  $Yes$  then
10:     $\text{REMEDIATEAGENT}(State.Failures)$  ▷ Tự động sửa lỗi
11:   end if
12: end if
13:  $PostScanData \leftarrow \text{RESCANAGENT}$  ▷ Quét lại kiểm chứng
14:  $Diff \leftarrow \text{ANALYSISAGENT}(State.RawFindings, PostScanData)$  ▷ So sánh trước/sau
15:  $P \leftarrow \text{REPORTAGENT}(Diff)$  ▷ Viết Markdown
16: return  $P$ 

```

2.3.7 Cơ chế Phản hồi và Suy luận (Reasoning Loop)

Để tránh việc hardcode (lập trình cứng) các quy tắc, các Agent sử dụng kỹ thuật **ReAct (Reasoning + Acting)**. Khi gặp một lỗi bảo mật:

1. **Thought (Suy nghĩ)**: Agent phân tích lỗi này thuộc dịch vụ nào (ví dụ: S3 Bucket public).
2. **Action (Hành động)**: Agent chọn công cụ tương ứng (ví dụ: `s3_block_public_access`).
3. **Observation (Quan sát)**: Agent kiểm tra kết quả trả về từ công cụ để xác nhận lỗi đã được sửa hay chưa trước khi báo cáo hoàn thành.

Related works and tools

Chương này phân tích các nền tảng công nghệ và công cụ mã nguồn mở được tích hợp trong hệ thống đề xuất. Việc lựa chọn các công cụ dựa trên các tiêu chí về khả năng mở rộng, tính riêng tư dữ liệu và hiệu suất thực thi trong môi trường cục bộ.

3.1 Công cụ Đánh giá Tư thế Bảo mật Đám mây (CSPM)

3.1.1 Prowler

Prowler là một công cụ dòng lệnh (CLI) mã nguồn mở tiêu chuẩn công nghiệp dùng để đánh giá bảo mật AWS. Nó được xây dựng dựa trên Khung kiến trúc tối ưu của AWS (AWS Well-Architected Framework) và các tiêu chuẩn CIS (Center for Internet Security).

Trong kiến trúc hệ thống Multi-Agent của đồ án, Prowler đóng vai trò là **Cảm biến (Sensor)**, cung cấp dữ liệu đầu vào thô cho quá trình nhận thức của tác nhân. Khác với các công cụ như AWS Security Hub (tốn chi phí) hay ScoutSuite (tập trung vào visualization), Prowler có khả năng xuất dữ liệu dưới định dạng **OCSF (Open Cybersecurity Schema Framework)**.

Định dạng OCSF giúp chuẩn hóa các phát hiện bảo mật (findings) thành cấu trúc JSON thống nhất, cho phép *Monitoring Agent* và *Remediate Agent* dễ dàng trích xuất các trường quan trọng như `event_code`, `resource_id`, và `status_code` mà không cần xử lý ngôn ngữ tự nhiên phức tạp.

[Sơ đồ: Prowler Architecture]
Prowler Core → AWS API Calls → Compliance Checks → JSON-OCSF Output

Hình 3.1: Luồng dữ liệu kiểm tra tuân thủ của Prowler

3.2 Hạ tầng Suy luận Mô hình Ngôn ngữ Cục bộ

3.2.1 Ollama và Llama 3.2

Để đảm bảo tính riêng tư dữ liệu—một yêu cầu tối thượng trong kiểm toán bảo mật—hệ thống sử dụng **Ollama** làm nền tảng suy luận cục bộ (Local Inference Runtime). Ollama giải quyết bài toán phức tạp của việc quản lý bộ nhớ và tải mô hình thông qua việc đóng gói các mô hình vào định dạng tiêu chuẩn.

Mô hình được lựa chọn là **Llama 3.2** (phiên bản 3B tham số). Đây là một Mô hình Ngôn ngữ Nhỏ (SLM) được tối ưu hóa thông qua kỹ thuật *Knowledge Distillation* và *Pruning* từ các mô hình lớn hơn (Llama 3.1 70B).

- **Cơ chế Attention:** Llama 3.2 sử dụng kiến trúc *Grouped-Query Attention (GQA)*, giúp giảm đáng kể băng thông bộ nhớ (VRAM) cần thiết trong quá trình giải mã (decoding), cho phép tốc độ sinh token nhanh hơn trên phần cứng giới hạn.
- **Instruction Tuning:** Mô hình đã được tinh chỉnh để tuân thủ các chỉ dẫn phức tạp, phù hợp cho việc phân tích ý định người dùng trong *Planning Agent* và tạo kế hoạch JSON trong *Remediate Agent*.

3.3 Quản lý Dữ liệu và Bộ nhớ (Data Serialization)

Hệ thống không sử dụng Database quan hệ hay Vector DB mà áp dụng cơ chế lưu trữ dựa trên tệp (File-based Storage) với định dạng ****JSON****, phù hợp với kiến trúc Micro-services cục bộ:

- **Tính cấu trúc:** JSON (JavaScript Object Notation) tương thích hoàn toàn với đầu ra của Prowler và đầu vào của LLM (thông qua LangChain Output Parsers).
- **Lưu vết (State Persistence):** Các tệp dữ liệu đóng vai trò là "Snapshots" của hệ thống tại từng thời điểm:
 - `pre_scan.json`: Trạng thái hạ tầng trước khi can thiệp.
 - `analysis_diff.json`: Bản ghi sự khác biệt (Delta) dùng để sinh báo cáo.
- **Hiệu năng:** Với quy mô quét cục bộ (Scoped Scan), việc đọc/ghi file JSON trực tiếp trên ổ cứng (I/O) nhanh hơn và ít tốn tài nguyên hơn đáng kể so với việc duy trì kết nối đến một Database Server.

3.4 AWS SDK cho Tự động hóa (Boto3)

Boto3 là bộ công cụ phát triển phần mềm (SDK) chính thức của AWS dành cho Python. Trong hệ thống này, Boto3 đóng vai trò là **Cơ cấu chấp hành (Actuator)**.

Các tác nhân AI không trực tiếp thay đổi môi trường đám mây. Thay vào đó, *Remediate Agent* tạo ra các tham số cấu hình, và các hàm *Tool* (được viết bằng Boto3) sẽ thực thi các thay đổi đó một cách an toàn. Boto3 cung cấp các cơ chế xử lý lỗi (Error Handling) và quản lý phiên (Session Management) ở mức thấp, giúp đảm bảo tính ổn định khi thực hiện các tác vụ nhạy cảm như `put_bucket_versioning` hay `put_user_policy`.

Chương này trình bày chi tiết kiến trúc kỹ thuật của hệ thống "Multi-Agent Security Auditor". Khác với các hệ thống rà soát truyền thống hoạt động theo luồng tuyến tính (Linear Scripting), giải pháp của chúng tôi được xây dựng dựa trên kiến trúc **Graph-based Orchestration** (Điều phối dựa trên Đồ thị) sử dụng framework LangGraph. Kiến trúc này cho phép quản lý trạng thái phức tạp, hỗ trợ các quy trình có tính chu trình (cyclic) và đảm bảo khả năng mở rộng module hóa.

4.1 Kiến trúc Điều phối Trung tâm (Graph-based Orchestration)

Trung tâm của hệ thống là bộ điều phối 'Graph Orchestrator', hoạt động như một máy trạng thái hữu hạn (Finite State Machine). Cấu trúc này định nghĩa luồng dữ liệu di chuyển giữa các Tác nhân (Agents) thông qua một bộ nhớ chia sẻ.

4.1.1 Quản lý Trạng thái (Shared State Management)

Thay vì truyền dữ liệu trực tiếp giữa các hàm, hệ thống duy trì một đối tượng trạng thái toàn cục (Global State Object) xuyên suốt vòng đời của phiên kiểm toán. Cấu trúc dữ liệu này được định nghĩa như sau:

$$\mathcal{S} = \{R_{user}, \mathcal{P}_{scan}, \mathcal{J}_{ids}, \mathcal{F}_{raw}, \mathcal{F}_{enriched}, \mathcal{F}_{fail}, \text{Flag}_{fix}\} \quad (4.1)$$

Trong đó, dựa trên mã nguồn thực tế (**AgentState TypedDict**):

- $\mathcal{P}_{scan}$: Kế hoạch quét được sinh ra bởi Planning Agent (JSON Config).
- $\mathcal{J}_{ids}$: Danh sách Job ID của các tiến trình Prowler đang chạy ngầm.
- $\mathcal{F}_{enriched}$: Danh sách các phát hiện (Findings) đã được Risk Agent làm giàu thông tin và chấm điểm.
- $\mathcal{F}_{fail}$: Tập hợp con các phát hiện có trạng thái "FAIL" cần khắc phục.
- Flag_{fix} : Cờ trạng thái (**remediation_needed**) xác định xem hệ thống có cần kích hoạt quy trình sửa lỗi hay không.

4.1.2 Sơ đồ Luồng xử lý (Workflow Pipeline)

Hệ thống được mô hình hóa thành một đồ thị có hướng (Directed Graph) với các đỉnh (Nodes) là các Agent chuyên biệt. Luồng thực thi tuân theo quy trình khép kín:

1. **Plan Node:** Phân tích yêu cầu người dùng → Tạo cấu hình quét.
2. **Scan Node:** Điều phối Scanner Agent gọi tool Prowler.

3. **Monitor Node:** Giám sát tiến trình và thu thập dữ liệu thô (Raw Findings).
4. **Risk Eval Node:** Đánh giá mức độ nghiêm trọng của từng lỗi.
5. **Assessment Node:** Lọc ra các lỗi vi phạm ngưỡng an toàn (Failures).
6. **Human Review Node:** (Điểm ngắt) Chờ xác nhận từ người dùng trước khi can thiệp hạ tầng.
7. **Remediate Node:** (Có điều kiện) Thực thi sửa lỗi nếu được duyệt.
8. **Rescan & Analysis Node:** Quét lại để kiểm chứng và so sánh sự thay đổi.
9. **Report Node:** Tổng hợp dữ liệu trạng thái cuối cùng để xuất báo cáo.

4.2 Cơ chế Hoạt động của các Tác nhân Chuyên biệt

Hệ thống áp dụng nguyên lý "Chuyên môn hóa Tác nhân", trong đó mỗi Agent sử dụng một chiến lược Prompting và cơ chế gọi Tool (Tool Usage) riêng biệt.

4.2.1 Scanner Agent: Dynamic Tool Binding

Scanner Agent chịu trách nhiệm tương tác với công cụ rà soát Prowler. Điểm cải tiến trong phiên bản này là cơ chế ****Dynamic Tool Binding**** thay vì Hardcoding.

- **Khởi tạo:** Agent tự động nạp danh sách công cụ từ module `agent_tools` và tạo ra một bảng ánh xạ động (`tools_map`).
- **Cơ chế gọi:** Sử dụng phương thức `bind_tools` của LangChain để gắn kèm định nghĩa hàm vào LLM. Khi nhận nhiệm vụ, Llama 3.2 sẽ tự động suy luận và trả về cấu trúc `tool_calls` chứa tên tool và tham số cần gọi.
- **Thực thi:** Agent tra cứu tên tool trong bảng ánh xạ và thực thi hàm tương ứng (ví dụ: `run_prowler_scan`), sau đó trích xuất Job ID để theo dõi.

4.2.2 Risk Evaluation Agent: JSON Mode & Scoring Rubric

Đây là tác nhân phân tích, chịu trách nhiệm chuyển đổi dữ liệu thô thành thông tin tình báo (Intelligence).

- **Input Processing:** Agent lọc danh sách các finding có trạng thái **FAIL** và loại bỏ các trường thông tin nhiễu.
- **Structured Output (JSON Mode):** Agent ép buộc mô hình ngôn ngữ trả về định dạng JSON nghiêm ngặt bằng tham số `format="json"`. Điều này đảm bảo tính nhất quán cho các bước xử lý máy học phía sau.
- **Scoring Rubric (Thang chấm điểm):** System Prompt chứa một bộ quy tắc chấm điểm rủi ro chi tiết (Rubric):
 - *Critical (9-10)*: Lỗi Public Access, lộ Key.
 - *High (7-8)*: Cấu hình sai nghiêm trọng nhưng giới hạn trong nội bộ.
 - *Medium (4-6)*: Thiếu Logging, Monitor.
 - *Low (1-3)*: Các lỗi về Tagging hoặc thông tin.
- **Nhiệm vụ:** Với mỗi finding, Agent sẽ đọc `description` và `status_details` để gán nhãn `severity` và `risk_score` chính xác.

4.2.3 Remediate Agent: Semantic Tool Selection

Tác nhân khắc phục sự cố được thiết kế theo hướng ****Fully Dynamic Inference****, cho phép hệ thống tự động tìm công cụ sửa lỗi phù hợp mà không cần lập trình trước các cặp "Lỗi - Cách sửa".

Quy trình suy luận diễn ra như sau:

1. **Context Loading:** Agent nạp toàn bộ danh sách mô tả của các công cụ sửa lỗi (ví dụ: `s3_enable_versioning`, `iam_rotate_key`) vào System Prompt.

2. **Semantic Matching:** Khi nhận được một lỗi (ví dụ: "S3 Bucket X thiếu Versioning"), LLM sẽ thực hiện so sánh ngữ nghĩa:

$$\text{Tool}_{\text{selected}} = \text{argmax}_{t \in \mathcal{T}}(\text{Similarity}(\text{Error_Desc}, t_{\text{desc}})) \quad (4.2)$$

Mô hình sẽ tự động hiểu rằng từ khóa "Versioning" trong lỗi khớp với công cụ `s3_enable_versioning`.

3. **Safety Guardrails:** Agent bao gồm logic kiểm tra an toàn:

- Tự động bỏ qua nếu `resource_id` là Account ID (để tránh tác động diện rộng).
- Chỉ thực thi khi độ tự tin (Confidence) của mô hình cao thông qua việc so khớp chính xác tên tool.

4.3 Quản lý Dữ liệu và Tính Bền vững (Data Persistence)

Do tính chất chạy cục bộ (Local Execution), hệ thống sử dụng chiến lược lưu trữ dựa trên tệp (File-based Storage) để đảm bảo hiệu năng cao và dễ dàng kiểm tra (audit).

- **Snapshots:** Tại các điểm chuyển giao quan trọng của đồ thị (Graph Nodes), dữ liệu được lưu xuống ổ cứng dưới dạng JSON:
 - `data/initial_scan_config.json`: Lưu cấu hình phạm vi quét.
 - `data/pre_scan.json`: Lưu trạng thái hệ thống trước khi can thiệp.
 - `data/analysis_diff.json`: Lưu kết quả so sánh (Diff) giữa trước và sau khi sửa lỗi.
- **Report Generation:** *Report Agent* đọc dữ liệu từ các tệp snapshot này để tổng hợp thành báo cáo Markdown cuối cùng, đảm bảo rằng báo cáo luôn phản ánh đúng dữ liệu thực tế đã xử lý.

Initial Implementation

Trong giai đoạn triển khai ban đầu, chúng tôi tập trung vào việc thiết lập nền tảng cốt lõi cho hệ thống Multi-Agent, bao gồm việc định nghĩa cấu trúc trạng thái (State Schema), chuẩn bị các công cụ tích hợp (Tool Binding), và cấu hình môi trường chạy cục bộ với Ollama và LangGraph.

Images/Implement/agent_workflow.png

Hình 5.1: Luồng dữ liệu trong hệ thống Multi-Agent

5.1 Data Structure Design (Thiết kế Cấu trúc Dữ liệu)

Khác với các hệ thống RAG truyền thống dựa vào Cơ sở dữ liệu Vector, hệ thống Agentic của chúng em hoạt động dựa trên luồng trạng thái (Stateful Workflow). Do đó, việc thiết kế cấu trúc dữ liệu tập trung vào đối tượng ****Shared State**** - bộ nhớ chia sẻ giữa các tác nhân.

5.1.1 Agent State Schema

Dựa trên thư viện **LangGraph**, chúng em định nghĩa một lược đồ trạng thái toàn cục (Global State Schema) để duy trì ngữ cảnh xuyên suốt phiên kiểm toán. Cấu trúc này được cài đặt dưới dạng Python TypedDict:

- **Input Data:**
 - **user_request:** Chuỗi văn bản chứa yêu cầu ban đầu của người dùng (ví dụ: "Scan S3 services").
 - **scan_plan:** Đối tượng JSON chứa cấu hình phạm vi quét do *Planning Agent* sinh ra.
- **Process Data:**
 - **job_ids:** Danh sách các định danh tiến trình (Process IDs) của Prowler đang chạy ngầm.
 - **raw_findings:** Dữ liệu thô thu được từ công cụ quét, chưa qua xử lý.
- **Output Intelligence:**
 - **enriched_findings:** Dữ liệu đã được *Risk Agent* làm giàu thông tin và chấm điểm rủi ro.
 - **failed_findings:** Danh sách lọc gọn các lỗi nghiêm trọng cần khắc phục.
 - **remediation_results:** Kết quả trả về từ các hành động sửa lỗi tự động.

Field Name	Data Type
scan_plan	Dict[str, Any]
enriched_findings	List[Dict]
remediation_needed	bool

Bảng 5.1: Các trường dữ liệu chính trong AgentState.

5.2 Data Preparation & Tooling (Chuẩn bị Công cụ)

Thay vì thu thập dữ liệu tĩnh, hệ thống Agent cần chuẩn bị các "giác quan" và "tay chân" để tương tác với môi trường AWS. Quá trình này bao gồm việc đóng gói các công cụ CLI và SDK thành các hàm Python có thể gọi được (Callable Functions).

Tool Definition Steps:

- **Prowler Integration:** Chúng tôi xây dựng một wrapper xung quanh công cụ Prowler để Agent có thể kích hoạt quét theo dòng lệnh.

```
1 def run_prowler_scan(service: str, region: str = "us-east-1"):
2     cmd = f"prowler aws --services {service} --region {region} --output-format json"
3
```

Listing 5.1: Prowler Tool Wrapper

- **Remediation Tools (Boto3):** Các hành động sửa lỗi được định nghĩa sẵn bằng thư viện boto3. Mỗi tool được gán một mô tả (docstring) chi tiết để LLM hiểu được ngữ nghĩa và cách sử dụng. Ví dụ: `s3_block_public_access`, `iam_rotate_access_key`.
- **Tool Registry:** Tất cả các công cụ được đăng ký vào một danh sách `SCANNER_AGENT_TOOLS` và `ALL_TOOLS` (trong file `agent_tools.py`) để phục vụ việc liên kết động (Dynamic Binding) với mô hình ngôn ngữ.

5.3 Data Preprocessing (Tiền xử lý Dữ liệu Quét)

Dữ liệu đầu ra từ Prowler thường rất lớn và chứa nhiều thông tin nhiễu kỹ thuật. Để LLM có thể xử lý hiệu quả, chúng tôi cài đặt quy trình tiền xử lý tại *Monitoring Agent* và *Risk Evaluation Agent*.

Key Preprocessing Steps:

- **Normalization (Chuẩn hóa):** Chuyển đổi các định dạng JSON khác nhau của Prowler (OCSF hoặc Native) về một cấu trúc phẳng (flat structure) thống nhất.
- **Filtering (Lọc nhiễu):** Loại bỏ các findings có trạng thái `PASS` hoặc `INFO`, chỉ giữ lại các findings có trạng thái `FAIL`. Logic này được thực hiện bằng mã nguồn Python thuần túy để giảm tải cho LLM và tăng tốc độ xử lý.
- **Field Extraction (Trích xuất trường):** Từ dữ liệu thô, chúng tôi chỉ trích xuất các trường quan trọng cần thiết cho việc ra quyết định:
 - `check_id`: Mã định danh lỗi (ví dụ: `s3_bucket_public_access`).
 - `resource_arn`: Định danh tài nguyên bị ảnh hưởng.
 - `status_extended`: Mô tả chi tiết nguyên nhân lỗi.

Đoạn mã dưới đây minh họa logic làm sạch dữ liệu trong `risk_evaluation_agent.py`:

```
1 def _clean_output_for_assessment(self, finding: dict, risk_data: dict) -> dict:
2     return {
3         "finding_id": finding.get("finding_id"),
4         "service_group": service_group,
5         "issue_hint": str(finding.get("prowler_check_id") or "").strip(),
6         "status": finding.get("status"),
7         "severity": risk_data.get("severity"),
8         "risk_score": risk_data.get("risk_score")
9     }
```

Listing 5.2: Finding Cleaning Logic

5.4 Orchestrator Setup (Thiết lập Bộ điều phối)

Thay vì thiết lập Database Server, "cơ sở dữ liệu" của hệ thống chính là Graph Workflow. Chúng tôi sử dụng **StateGraph** của LangGraph để kết nối các Agent.

5.4.1 Node & Edge Definition

Hệ thống được khởi tạo bằng cách đăng ký các hàm xử lý (Node) và định nghĩa luồng đi (Edge):

- **Nodes:** Bao gồm `plan_node`, `scan_node`, `monitor_node`, `risk_eval_node`, `remediate_node`, v.v. Mỗi node là một hàm Python nhận vào `AgentState` và trả về phần cập nhật (State Update).
- **Conditional Edges:** Logic rẽ nhánh quan trọng được cài đặt tại `human_approval_node`.

$$\text{Next Node} = \begin{cases} \text{remediate} & \text{if approved} == \text{True} \\ \text{report} & \text{if approved} == \text{False} \end{cases}$$

5.4.2 File-based Persistence

Để đảm bảo tính bền vững (Persistence) mà không cần DB phức tạp, hệ thống lưu các snapshot trạng thái xuống thư mục `data/` dưới dạng JSON (`pre_scan.json`, `post_scan.json`).

5.5 Local LLMs Setup (Cấu hình Mô hình Ngôn ngữ)

Chúng tôi sử dụng **Ollama** làm nền tảng chạy mô hình ngôn ngữ cục bộ, đảm bảo dữ liệu nhạy cảm về hạ tầng AWS không bị gửi ra ngoài Internet.

5.5.1 Model Selection Configuration

Mô hình được lựa chọn là `llama3.2` (phiên bản 3B parameters) nhờ khả năng cân bằng giữa tốc độ và khả năng suy luận (Reasoning).

```
OLLAMA_MODEL = "llama3.2"
OLLAMA_BASE_URL = "http://localhost:11434"
```

5.5.2 Integration with Agents

Việc tích hợp LLM vào từng Agent được thực hiện thông qua thư viện `langchain_ollama`. Chúng tôi áp dụng hai chiến lược cấu hình khác nhau tùy thuộc vào vai trò của Agent:

1. **JSON Mode (cho Risk Evaluation Agent):** Cấu hình `format="json"` và `temperature=0` để đảm bảo đầu ra luôn là cấu trúc dữ liệu máy có thể đọc được, phục vụ cho việc chấm điểm rủi ro.
2. **Tool Calling Mode (cho Scanner & Remediate Agent):** Sử dụng phương thức `bind_tools` để cho phép mô hình nhận biết và lựa chọn công cụ từ danh sách `agent_tools`.

```
1 self.llm = ChatOllama(model=model_name, temperature=0)
2 self.llm_with_tools = self.llm.bind_tools(SCANNER_AGENT_TOOLS)
```

Listing 5.3: LLM Initialization with Tools

Cấu hình này cho phép hệ thống vận hành trơn tru trên máy cá nhân với RAM 8GB-16GB mà vẫn đảm bảo khả năng xử lý các tác vụ phức tạp.

6.1 Overview

Tính đến tháng 12 năm 2025, các nhiệm vụ nghiên cứu cơ sở và triển khai lõi hệ thống (Core Implementation) đã hoàn tất. Giai đoạn tiếp theo sẽ tập trung vào việc nâng cao khả năng suy luận bằng RAG, mở rộng bộ công cụ sửa lỗi và tối ưu hóa trải nghiệm người dùng.

6.2 Current tasks: Early December 2025

Hiện tại, chúng em đang hoàn tất giai đoạn Initial Implementation. Các công việc chính bao gồm:

- Ổn định hóa luồng hoạt động của *Graph Orchestrator*.
- Kiểm tra khả năng gọi tool động (Dynamic Tool Binding) của *Scanner Agent* và *Remediate Agent* với mô hình Llama 3.2.
- Thu thập dữ liệu log ban đầu để chuẩn bị cho việc xây dựng kịch bản kiểm thử.

6.3 Upcoming tasks: January 2026 and Beyond

Dựa trên kết quả hiện tại, lộ trình phát triển chi tiết cho học kỳ tới như sau:

1. Building Knowledge Base & RAG Integration (January 2025):

- Xây dựng cơ sở dữ liệu Vector (sử dụng ChromaDB hoặc Milvus) chứa các tiêu chuẩn bảo mật (CIS Benchmarks, AWS Well-Architected).
- Tích hợp cơ chế truy xuất (Retrieval) vào *Assessment Agent* để giảm thiểu ảo giác và tăng tính chính xác khi giải thích lỗi.

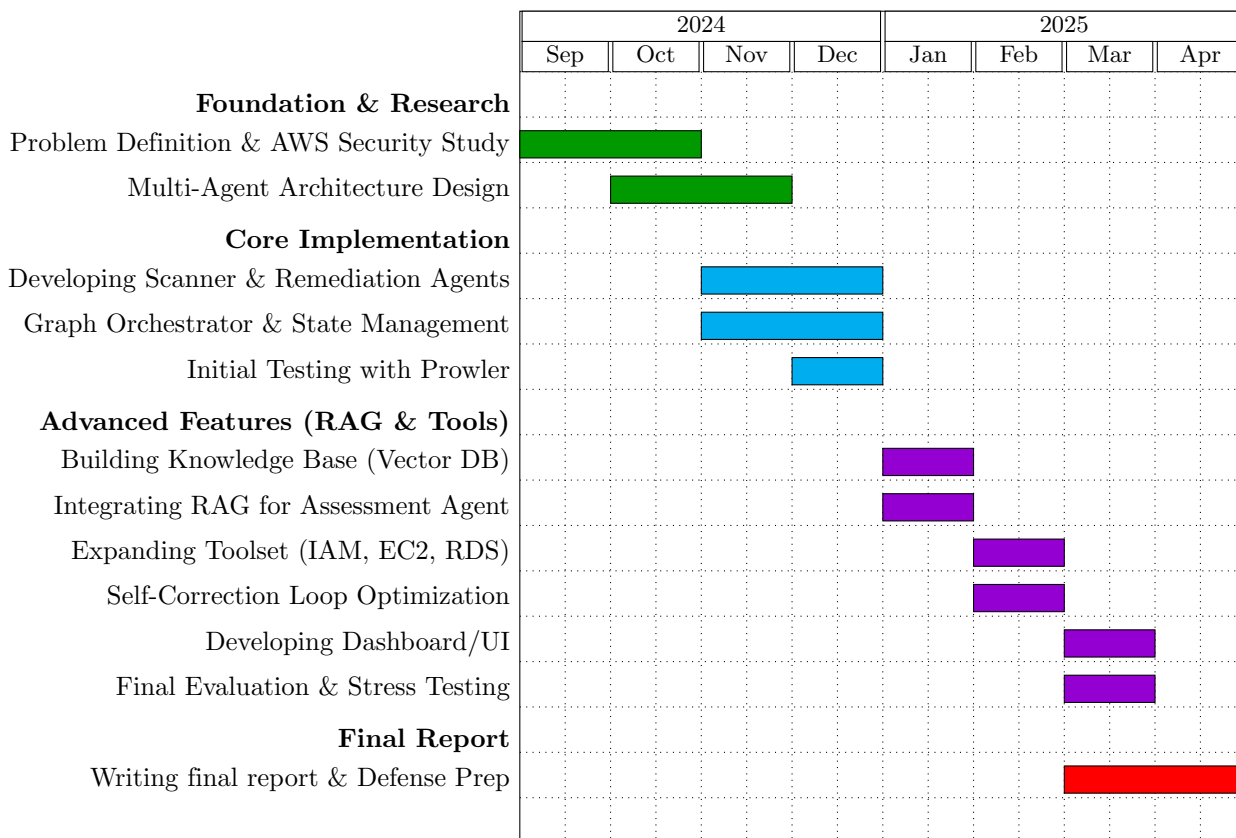
2. Expanding Remediation Toolset (February 2025):

- Mở rộng thư viện công cụ (`agent_tools.py`) ra ngoài phạm vi S3.
- Phát triển các modules khắc phục cho IAM (xoá user không hoạt động, enforce MFA) và EC2 (đóng Security Group port 22/3389).
- Kiểm thử tính an toàn của các tool mới trong môi trường Sandbox.

3. Self-Correction & System Optimization (February 2025):

- Tối ưu hóa logic tự sửa lỗi (Self-Correction Loop) để giảm số bước thực thi.
- Tinh chỉnh System Prompt cho Llama 3.2 để cải thiện tốc độ phản hồi và định dạng JSON đầu ra.

4. Developing User Interface (March 2025):



Hình 6.1: Project Timeline and Future Plan

- Xây dựng giao diện người dùng (Web Dashboard hoặc Streamlit) để trực quan hóa biểu đồ trạng thái (State Graph) và báo cáo bảo mật.
- Tích hợp tính năng "Human-in-the-loop" trực quan trên giao diện thay vì CLI.

5. Final Evaluation & Deployment (March - April 2025):

- Thực hiện đánh giá toàn diện dựa trên các chỉ số đã đề ra (Tool Selection Accuracy, Remediation Success Rate).
- Đóng gói ứng dụng (Dockerize) để sẵn sàng triển khai thực tế.

6. Writing Final Report (April 2025):

- Tổng hợp kết quả nghiên cứu, so sánh hiệu năng trước và sau khi tích hợp RAG.
- Hoàn thiện báo cáo khóa luận và chuẩn bị slide bảo vệ.

Bibliography

- [1] Jinze Bai et al. “Qwen Technical Report”. In: *arXiv preprint* arXiv:2309.16609 (2023). Available at <https://arxiv.org/abs/2309.16609>.
- [2] Jianguo Wang et al. “Milvus: A Purpose-Built Vector Data Management System”. In: *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Virtual Event, China, 2021, pp. 2624–2638. DOI: 10.1145/3448016.3457550.